SCHOOL MANAGEMENT SYSTEM

Database Design Document

Prepared by: Development Team
Client: School Representative
Version: 1.0 (Draft)
Date: March 2026

1. Introduction

1.1 Purpose

This Database Design Document (DDD) defines the complete relational database schema for the School Management System (SMS). It specifies every table, column, data type, constraint, relationship, and index required to store and manage all school data reliably, securely, and efficiently. It is the authoritative reference for backend development and serves as the contract between the database layer and the application service layer.

1.2 Scope

This document covers the full database schema for all six SMS modules:

- Module 1: Pupil Information Management
- Module 2: School Fees Billing and Payment Management
- Module 3: Parent Communication and Follow-Up Management
- Module 4: Academic Performance Management
- Module 5: Parent Portal (schema future-proofed for upcoming release)
- Module 6: System Administration, Configuration, and Audit

1.3 Document Overview

Attribute	Detail
Project	School Management System (SMS)
Document Type	Database Design Document (DDD)
Version	1.0 (Draft)
Database Engine	PostgreSQL 15+
ORM	Prisma ORM v5+
Based On	RAD v1.1 · System Architecture v1.0 · UI/UX Design v1.0 · Report Card Samples
Date	March 2026
Author	Development Team

2. Database Design Principles

The following principles govern every design decision in this schema. They are non-negotiable and must be respected throughout development and any future schema changes.

Principle	Application in This Schema
UUID Primary Keys	All tables use UUID v4 as their primary key. This prevents sequential ID enumeration, supports future data migration, and enables offline ID generation on the client.
Soft Deletion	Pupil, guardian, user, and class records are never hard-deleted. A deleted_at TIMESTAMPTZ column (null = active, non-null = deleted) is used throughout. Queries always filter WHERE deleted_at IS NULL.
Temporal Audit Columns	Every table includes created_at and updated_at TIMESTAMPTZ columns, managed automatically by Prisma. Key tables also include created_by UUID referencing users.
Integer Currency	All monetary amounts are stored as INTEGER in the smallest currency unit (Uganda Shillings have no decimal subdivision). This eliminates floating-point rounding errors in financial calculations.
Nullable vs Zero Scores	A pupil's exam score is stored as SMALLINT NULL. NULL means the exam was not sat (missed). Zero (0) means the pupil sat and scored zero. These are semantically different and must not be conflated.
Configurable Rules	No grading boundaries, fee structures, subject lists, or report card settings are hardcoded in the schema. All are stored as data rows in configuration tables, readable and editable by the System Administrator at runtime.
Append-Only Audit Log	The audit_logs table is append-only. No row is ever updated or deleted from it. It provides a tamper-evident record of all significant system actions.
Referential Integrity	All foreign key relationships are enforced at the database level with explicit ON DELETE RESTRICT or ON DELETE CASCADE rules, preventing orphaned records.
Normalisation	The schema is normalised to Third Normal Form (3NF) to eliminate data redundancy. Computed values (aggregates, divisions, ranks) are stored in result tables after calculation — not recalculated on every query.
Terminology	A learner in Pre-Primary or Primary (P.1–P.7) is a PUPIL throughout this schema. Column names, table names, and comments use 'pupil' not 'student'.

3. Entity Groups Overview

The schema is organised into eight logical entity groups. Each group contains all tables relevant to a particular domain. The groups map directly to the SMS modules.

#	Group	Tables	Purpose
A	**Users & Access**	`users, roles, user_roles, sessions, audit_logs`	System user accounts, role-based access control, session tracking, and full audit trail.
B	**School Configuration**	`school_settings, academic_years, terms, school_sections, classes, streams`	Core school identity, academic calendar, class structure, and section definitions.
C	**Subjects & Grading**	`subjects, class_subject_assignments, subject_section_rules, grading_scales, grading_scale_entries, assessment_type_configs`	Configurable subject assignments per class, per-section aggregate rules, admin-defined grading scale, and assessment type management.
D	**Pupils & Families**	`pupils, guardians, pupil_guardians, pupil_photos`	Central pupil registry, guardian profiles, family-linking, and pupil photo storage.
E	**Fees**	`fee_categories, fee_structures, bursary_schemes, pupil_bursaries, pupil_bills, bill_line_items, payments, payment_plans, payment_plan_instalments, fee_statements`	Complete fees lifecycle: category and structure configuration, bursary management, per-pupil billing, payment recording, instalment plans, and printable statements.
F	**Academic Performance**	`assessment_periods, pupil_scores, pupil_term_results, report_cards, report_card_settings, term_requirements`	Score entry, automated grading engine output, report card generation records, and configurable report card settings.
G	**Communication**	`communication_logs, message_templates, demand_notes`	Full parent interaction history, message templates, and demand note records.
H	**System**	`system_settings, scheduled_job_logs`	School-wide configurable settings and background job execution records.

4. Complete Table Schema

> **Convention** Column names use snake_case. PK = Primary Key. FK = Foreign Key. UQ = Unique. NN = Not Null. DEF = Default value. All UUIDs generated via gen_random_uuid(). All timestamps stored as TIMESTAMPTZ (timezone-aware).

GROUP A — USERS & ACCESS
users · roles · user_roles · sessions · audit_logs

A1. users

Stores all system user accounts. Each user has exactly one role. Passwords are stored as bcrypt hashes — never plaintext.

Column	Data Type	Constraints	Description
id	UUID	PK, NN, DEF gen_random_uuid()	Unique system identifier for the user.
full_name	VARCHAR(150)	NN	Full name of the staff member or parent.
username	VARCHAR(100)	UQ, NN	Login username. Lowercase, no spaces.
email	VARCHAR(200)	UQ	Email address. Used for password reset.
password_hash	VARCHAR(255)	NN	bcrypt hash of the user's password.
role_id	UUID	FK → roles.id, NN	The role assigned to this user.
is_active	BOOLEAN	NN, DEF true	False = account deactivated. User cannot log in.
last_login_at	TIMESTAMPTZ	NULL	Timestamp of the most recent successful login.
must_change_password	BOOLEAN	NN, DEF false	True = user must change password on next login.
deleted_at	TIMESTAMPTZ	NULL	Soft delete. NULL = active.
created_at	TIMESTAMPTZ	NN, DEF now()	Record creation timestamp.
updated_at	TIMESTAMPTZ	NN, DEF now()	Last update timestamp.

A2. roles

Defines the available user roles in the system. Roles are seeded at deployment and managed by the System Administrator.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	Unique role identifier.
name	VARCHAR(80)	UQ, NN	Role name. E.g. system_admin, head_teacher, bursar, class_teacher, dos, parent.
display_name	VARCHAR(120)	NN	Human-readable role label. E.g. 'Director of Studies'.
description	TEXT	NULL	Optional description of the role's responsibilities.
created_at	TIMESTAMPTZ	NN, DEF now()	

> **Roles Seeded at Deployment** system_admin · head_teacher · bursar · class_teacher · dos (Director of Studies) · parent

A3. user_roles — Role Permission Reference

Supplementary table that maps roles to named permission strings, enabling fine-grained feature-level access control beyond the base role.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
role_id	UUID	FK → roles.id, NN	The role this permission applies to.
permission	VARCHAR(120)	NN	Permission string. E.g. 'scores.edit', 'report_cards.generate', 'fees.discount.apply'.
created_at	TIMESTAMPTZ	NN, DEF now()	

A4. sessions

Tracks active authenticated sessions via Refresh Tokens. Revoked on logout or expiry.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
user_id	UUID	FK → users.id, NN	The authenticated user.
refresh_token_hash	VARCHAR(255)	UQ, NN	Hashed refresh token. Never stored in plain text.
ip_address	VARCHAR(45)	NULL	IPv4 or IPv6 address of the client at login.
user_agent	TEXT	NULL	Browser/client user agent string.

expires_at	TIMESTAMPTZ	NN	When this refresh token expires.
revoked_at	TIMESTAMPTZ	NULL	Set on logout. NULL = session still valid.
created_at	TIMESTAMPTZ	NN, DEF now()	

A5. audit_logs

Append-only record of all significant system actions. Never updated or deleted.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
user_id	UUID	FK → users.id, NULL	The user who performed the action. NULL for system-triggered events.
action	VARCHAR(120)	NN	Action code. E.g. 'PUPIL_CREATED', 'PAYMENT_RECORDED', 'SCORE_EDITED', 'REPORT_CARD_GENERATED'.
entity_type	VARCHAR(80)	NN	The affected entity type. E.g. 'pupil', 'payment', 'score'.
entity_id	UUID	NULL	The UUID of the affected record.
old_value	JSONB	NULL	Previous state of the record (for updates). Stored as JSON.
new_value	JSONB	NULL	New state of the record (for creates/updates). Stored as JSON.
ip_address	VARCHAR(45)	NULL	Client IP address at the time of the action.
notes	TEXT	NULL	Optional contextual note for the audit entry.
created_at	TIMESTAMPTZ	NN, DEF now()	Timestamp of the action.

GROUP B — SCHOOL CONFIGURATION

school_settings · academic_years · terms · school_sections · classes · streams

B1. school_settings

Single-row table holding the school's identity and configurable system-wide settings. Managed exclusively by the System Administrator.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	Always a single row.
school_name	VARCHAR(200)	NN	Full official school name.
school_motto	VARCHAR(200)	NULL	School motto displayed on reports and UI.
address_line1	VARCHAR(200)	NULL	Physical address line 1.
address_line2	VARCHAR(200)	NULL	Physical address line 2 / PO Box.
phone_primary	VARCHAR(30)	NULL	Primary contact phone number.
phone_secondary	VARCHAR(30)	NULL	Secondary contact phone number.
email	VARCHAR(200)	NULL	School email address.
logo_path	VARCHAR(500)	NULL	File path or URL to the school crest/logo image.
stamp_notice	TEXT	NULL	Footer notice on report cards e.g. 'Not valid without official school stamp'.
report_footer_motto	VARCHAR(200)	NULL	Motivational quote shown at the bottom of report cards.
grade_guide_text	TEXT	NULL	Configurable grade guide line shown on report cards. E.g. '90-100=D1, 80-89=D2 ...'
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

B2. academic_years

Represents a single academic year. All terms, fees, and results are scoped to an academic year.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
year_label	VARCHAR(20)	UQ, NN	E.g. '2026'. Used as display label throughout the system.

start_date	DATE	NN	Official start date of the academic year.
end_date	DATE	NN	Official end date of the academic year.
is_current	BOOLEAN	NN, DEF false	Only one academic year can be current at a time. Enforced by application logic.
created_by	UUID	FK → users.id, NULL	Admin who created this record.
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

B3. terms

Represents a single school term within an academic year. Holds all term-specific dates including separate Day and Boarding next-term start dates for the report card footer.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
academic_year_id	UUID	FK → academic_years.id, NN	The academic year this term belongs to.
term_number	SMALLINT	NN	1, 2, or 3.
term_label	VARCHAR(40)	NN	Display label. E.g. 'Term 1, 2026'.
start_date	DATE	NN	Term start date.
end_date	DATE	NN	Term end date.
next_term_start_day	DATE	NULL	Next term start date for Day pupils (printed on report card).
next_term_start_boarding	DATE	NULL	Next term start date for Boarding pupils (printed on report card).
is_current	BOOLEAN	NN, DEF false	Only one term can be current at a time.
fees_due_date	DATE	NULL	Default fees payment due date for this term.
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

B4. school_sections

Defines the two primary school sections. Each section has its own ranking method and aggregate inclusion rules.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
name	VARCHAR(80)	UQ, NN	E.g. 'Lower Primary', 'Upper Primary'.
code	VARCHAR(20)	UQ, NN	Short code. E.g. 'LOWER', 'UPPER'.
ranking_method	VARCHAR(20)	NN, DEF 'aggregate'	'aggregate' (upper primary) or 'average' (lower primary).
classes_description	VARCHAR(100)	NULL	Informational. E.g. 'P.1 – P.3'.
created_at	TIMESTAMPTZ	NN, DEF now()	

B5. classes

Represents a class level (e.g. P.5). Each class belongs to a school section. Classes are configured by the System Administrator.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
school_section_id	UUID	FK → school_sections.id, NN	The section this class belongs to (Lower or Upper Primary).
name	VARCHAR(40)	NN	Class name. E.g. 'P.5'.
level_order	SMALLINT	NN	Numeric order for sorting. E.g. 1 for Baby Class, 7 for P.7.
is_active	BOOLEAN	NN, DEF true	Inactive classes are hidden from assignment dropdowns.
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

B6. streams

A stream is a named division within a class (e.g. 'Crane', 'Eagle'). A class can have one or more streams. Each stream has one assigned class teacher per term.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	

class_id	UUID	FK → classes.id, NN	The class this stream belongs to.
name	VARCHAR(60)	NN	Stream name. E.g. 'Crane', 'Eagle'.
academic_year_id	UUID	FK → academic_years.id, NN	Streams are re-configured each academic year.
class_teacher_id	UUID	FK → users.id, NULL	The class teacher assigned to this stream for the academic year.
is_active	BOOLEAN	NN, DEF true	
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

GROUP C — SUBJECTS & GRADING

subjects · class_subject_assignments · subject_section_rules · grading_scales · grading_scale_entries · assessment_type_configs

C1. subjects

Master list of all subjects the school can offer. Which subjects a class actually takes is defined in class_subject_assignments.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
name	VARCHAR(120)	UQ, NN	Subject name. E.g. 'English', 'Mathematics', 'Literacy I'.
code	VARCHAR(20)	UQ, NN	Short code. E.g. 'ENG', 'MATH', 'LIT1'.
is_active	BOOLEAN	NN, DEF true	Inactive subjects cannot be assigned to classes.
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

C2. class_subject_assignments

Defines exactly which subjects a specific class takes in a specific term. Only assigned subjects appear on the report card. This replaces all hardcoded subject lists.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
class_id	UUID	FK → classes.id, NN	The class receiving this subject.
subject_id	UUID	FK → subjects.id, NN	The subject being assigned.
term_id	UUID	FK → terms.id, NN	The term this assignment is active for.
display_order	SMALLINT	NN, DEF 1	Order in which the subject appears on the report card.
max_score	SMALLINT	NN, DEF 100	Maximum achievable score for this subject in this class.
created_by	UUID	FK → users.id, NULL	Admin who created this assignment.
created_at	TIMESTAMPTZ	NN, DEF now()	

UNIQUE		(class_id, subject_id, term_id)	A subject can only be assigned once per class per term.

C3. subject_section_rules

Defines per-section rules for each subject — specifically whether the subject is included in the aggregate or average calculation. This is what enforces the 'Religious Education excluded from aggregate' rule and all similar rules, without hardcoding.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
school_section_id	UUID	FK → school_sections.id, NN	The section this rule applies to.
subject_id	UUID	FK → subjects.id, NN	The subject this rule applies to.
include_in_aggregate	BOOLEAN	NN, DEF true	False = excluded from aggregate/average calculation. E.g. Religious Education = false.
is_penalty_trigger	BOOLEAN	NN, DEF false	True = F9 in this subject triggers automatic division demotion. E.g. English = true, Mathematics = true.
created_at	TIMESTAMPTZ	NN, DEF now()	
UNIQUE		(school_section_id, subject_id)	One rule per subject per section.

C4. grading_scales

A named grading scale that can be activated by the System Administrator. Multiple scales can exist but only one is active at a time. This allows the admin to change grade boundaries (e.g. D1 from 90-100 to 85-100) without code changes.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
name	VARCHAR(100)	NN	Name of the scale. E.g. 'Standard Uganda MoES Scale', 'Modified 2026 Scale'.
is_active	BOOLEAN	NN, DEF false	Only one scale can be active. Enforced by application logic before save.
description	TEXT	NULL	Optional notes on why this scale was created or when it applies.

created_by	UUID	FK → users.id, NULL	
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

C5. grading_scale_entries

The individual grade bands within a grading scale. Each row defines one grade. The grading engine reads these rows at runtime to compute grade labels and points from raw scores.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
grading_scale_id	UUID	FK → grading_scales.id, NN	The scale this entry belongs to.
grade_label	VARCHAR(10)	NN	Grade label. E.g. 'D1', 'D2', 'C3', 'P7', 'F9'.
points_value	SMALLINT	NN	Points awarded for this grade. E.g. D1=1, D2=2, F9=9.
min_score	SMALLINT	NN	Minimum score (inclusive) for this grade band.
max_score	SMALLINT	NN	Maximum score (inclusive) for this grade band.
display_order	SMALLINT	NN	Render order on the grade guide (1=best, 9=worst).
is_fail	BOOLEAN	NN, DEF false	True = this grade is a failing grade (F9).
UNIQUE		(grading_scale_id, grade_label)	A label can only appear once per scale.
UNIQUE		(grading_scale_id, points_value)	A points value can only appear once per scale.

> **Grading Engine Rule** The grading engine always queries the single active grading_scale and its entries to compute grade labels and points. Changing the active scale or its entries immediately affects all future score computations.

C6. division_boundaries

Defines the aggregate point ranges that correspond to each Division. These are also admin-configurable, separate from per-subject grading.

Column	Data Type	Constraints	Description

id	UUID	PK, NN	
grading_scale_id	UUID	FK → grading_scales.id, NN	The scale these boundaries belong to.
division_label	VARCHAR(20)	NN	E.g. 'I', 'II', 'III', 'IV', 'U'.
roman_numeral	VARCHAR(10)	NN	Roman numeral display. E.g. 'I', 'II', 'III', 'IV', 'U'.
min_aggregate	SMALLINT	NN	Minimum aggregate points for this division (lower is better).
max_aggregate	SMALLINT	NN	Maximum aggregate points for this division.
is_ungraded	BOOLEAN	NN, DEF false	True = this is the fail/ungraded band (35-36 points).
display_order	SMALLINT	NN	Render order (1=Division I).

C7. assessment_type_configs

Defines all assessment types available in the system. BOT, MOT, and EOT are seeded. Additional types (e.g. PLE Mock, Topical Test) can be added by the System Administrator at any time.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
code	VARCHAR(30)	UQ, NN	Internal code. E.g. 'BOT', 'MOT', 'EOT', 'PLE_MOCK', 'TOPICAL'.
label	VARCHAR(80)	NN	Display name. E.g. 'Beginning of Term', 'PLE Mock Examination'.
appears_on_report_card	BOOLEAN	NN, DEF true	True = this assessment type's scores appear on the report card.
contributes_to_aggregate	BOOLEAN	NN, DEF false	True = this assessment's score contributes to the term aggregate. Typically only EOT does.
display_order	SMALLINT	NN	Column order on the report card.
is_system_default	BOOLEAN	NN, DEF false	True = BOT/MOT/EOT. Cannot be deleted, only deactivated.
is_active	BOOLEAN	NN, DEF true	Inactive types are hidden from score entry screens.

created_by	UUID	FK → users.id, NULL	NULL for system-seeded defaults.
created_at	TIMESTAMPTZ	NN, DEF now()	

GROUP D — PUPILS & FAMILIES

pupils · guardians · pupil_guardians · pupil_photos

D1. pupils

The central pupil registry. Every enrolled pupil has exactly one record in this table. This is the most critical table in the schema.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	Unique internal pupil identifier.
pupil_id_code	VARCHAR(30)	UQ, NN	System-generated internal ID. E.g. 'SMS-2024-001'. Format configurable in school_settings.
lin	VARCHAR(50)	UQ, NULL	Learner Identification Number issued by Uganda MoES. Nullable as new pupils may not have one yet.
schoolpay_code	VARCHAR(50)	UQ, NULL	SchoolPay payment code assigned by the SchoolPay platform.
first_name	VARCHAR(80)	NN	
last_name	VARCHAR(80)	NN	
other_names	VARCHAR(80)	NULL	Middle names or other names.
date_of_birth	DATE	NN	
gender	VARCHAR(10)	NN	'Male' or 'Female'.
religion	VARCHAR(60)	NULL	E.g. 'Catholic', 'Muslim', 'Protestant'.
house	VARCHAR(60)	NULL	School house assignment if applicable.
medical_conditions	TEXT	NULL	Any known medical conditions or allergies.
former_school	VARCHAR(200)	NULL	Previous school attended before enrolment.
stream_id	UUID	FK → streams.id, NULL	The stream the pupil is currently enrolled in.
section	VARCHAR(20)	NN	'Day' or 'Boarding'.
enrolment_date	DATE	NN	Date the pupil was enrolled at this school.
is_active	BOOLEAN	NN, DEF true	False = withdrawn or transferred pupil.

deleted_at	TIMESTAMPTZ	NULL	Soft delete. NULL = active.
created_by	UUID	FK → users.id, NULL	Staff member who registered this pupil.
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

D2. guardians

Parent or guardian profiles. A single guardian can be linked to multiple pupils (siblings). Storing the guardian once and linking avoids data duplication across siblings.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
full_name	VARCHAR(150)	NN	
relationship	VARCHAR(60)	NN	E.g. 'Father', 'Mother', 'Uncle', 'Legal Guardian'.
phone_call	VARCHAR(30)	NN	Primary phone number used for voice calls.
phone_whatsapp	VARCHAR(30)	NULL	WhatsApp number (may differ from call number).
email	VARCHAR(200)	NULL	
physical_address	TEXT	NULL	Home or work physical address.
occupation	VARCHAR(100)	NULL	Optional. Useful for contextual communication.
is_active	BOOLEAN	NN, DEF true	
deleted_at	TIMESTAMPTZ	NULL	Soft delete.
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

D3. pupil_guardians

Junction table linking pupils to their guardians. A pupil can have multiple guardians (e.g. both parents). One guardian is designated as primary for communication purposes.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	

guardian_id	UUID	FK → guardians.id, NN	
is_primary_contact	BOOLEAN	NN, DEF false	True = this guardian is the first contact for fees follow-up and communication.
created_at	TIMESTAMPTZ	NN, DEF now()	
UNIQUE		(pupil_id, guardian_id)	A guardian is linked to a pupil only once.

D4. pupil_photos

Stores the file reference for a pupil's passport photo. Separated from the main pupils table to keep the core record lightweight. If no photo exists, the UI renders initials.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, UQ, NN	One photo record per pupil.
file_path	VARCHAR(500)	NN	Server file path or object storage key for the photo file.
file_size_bytes	INTEGER	NULL	File size for storage management.
uploaded_by	UUID	FK → users.id, NULL	
uploaded_at	TIMESTAMPTZ	NN, DEF now()	

GROUP E — FEES

fee_categories · fee_structures · bursary_schemes · pupil_bursaries · pupil_bills · bill_line_items · payments · payment_plans · payment_plan_instalments · fee_statements

E1. fee_categories

Admin-defined fee category master list. E.g. Tuition, Transport, UNEB, Development. Categories are reused across terms.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
name	VARCHAR(120)	UQ, NN	E.g. 'Tuition Fees', 'Transport Fees', 'UNEB Examination Fees'.
code	VARCHAR(30)	UQ, NN	Short code. E.g. 'TUITION', 'TRANSPORT', 'UNEB'.
is_tuition	BOOLEAN	NN, DEF false	True = this is the core tuition category. Used for bursary and billing logic.
is_active	BOOLEAN	NN, DEF true	
created_at	TIMESTAMPTZ	NN, DEF now()	

E2. fee_structures

Defines the standard fee amount for a specific category, class, section, and term. This is the school's published fee schedule. Since fees differ by class and section (Day vs Boarding), each combination has its own row.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
fee_category_id	UUID	FK → fee_categories.id, NN	The fee category this structure entry applies to.
class_id	UUID	FK → classes.id, NN	The class this fee applies to.
section	VARCHAR(20)	NN	'Day' or 'Boarding'. Tuition differs by section.
term_id	UUID	FK → terms.id, NN	The term this fee structure is valid for.
amount	INTEGER	NN	Standard fee amount in Uganda Shillings (integer, no decimals).
is_mandatory	BOOLEAN	NN, DEF true	False = optional fee (e.g. transport for day pupils only).

created_by	UUID	FK → users.id, NULL	
created_at	TIMESTAMPTZ	NN, DEF now()	
UNIQUE		(fee_category_id, class_id, section, term_id)	One fee per category per class per section per term.

E3. bursary_schemes

Admin-defined bursary types. A bursary is a structured discount scheme with a named type. Separate from ad-hoc discounts.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
name	VARCHAR(150)	UQ, NN	E.g. 'Full Bursary', 'Half Bursary', 'Staff Child Discount', '50% Sibling Discount'.
discount_type	VARCHAR(20)	NN	'percentage' or 'fixed_amount'.
discount_value	INTEGER	NN	If percentage: 0–100. If fixed_amount: amount in UGX.
applies_to_category_id	UUID	FK → fee_categories.id, NULL	NULL = applies to all categories. Non-null = applies to one specific category only.
is_active	BOOLEAN	NN, DEF true	
created_at	TIMESTAMPTZ	NN, DEF now()	

E4. pupil_bursaries

Links a pupil to a bursary scheme. A pupil can have at most one active bursary at a time. This drives automatic discount calculation during bulk billing.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	
bursary_scheme_id	UUID	FK → bursary_schemes.id, NN	
awarded_date	DATE	NN	Date the bursary was formally awarded.
expiry_date	DATE	NULL	Optional expiry. NULL = indefinite.

notes	TEXT	NULL	Context or reason for the bursary award.
awarded_by	UUID	FK → users.id, NULL	
is_active	BOOLEAN	NN, DEF true	
created_at	TIMESTAMPTZ	NN, DEF now()	

E5. pupil_bills

The top-level fees bill for a single pupil for a single term. Generated by the bulk billing process or individually by the Bursar.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	
term_id	UUID	FK → terms.id, NN	The term this bill is for.
total_amount	INTEGER	NN	Total amount billed (sum of line items after discounts). In UGX.
total_paid	INTEGER	NN, DEF 0	Running total of all payments recorded. Updated on each payment.
balance	INTEGER	NN	Computed: total_amount - total_paid. Updated on each payment.
arrears_from_previous	INTEGER	NN, DEF 0	Outstanding balance carried over from the previous term.
billing_status	VARCHAR(20)	NN, DEF 'pending'	'pending', 'partial', 'paid', 'overdue'.
generated_by	UUID	FK → users.id, NULL	Staff member or system process that generated this bill.
generated_at	TIMESTAMPTZ	NN, DEF now()	
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	
UNIQUE		(pupil_id, term_id)	A pupil has only one bill per term.

E6. bill_line_items

Individual line items within a pupil's bill. One row per fee category. Stores the standard amount, bursary discount applied, and the net amount charged.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_bill_id	UUID	FK → pupil_bills.id, NN	The parent bill this line item belongs to.
fee_category_id	UUID	FK → fee_categories.id, NN	The fee category for this line item.
standard_amount	INTEGER	NN	Amount from the fee_structures table before any discount.
bursary_discount	INTEGER	NN, DEF 0	Discount amount applied (from pupil_bursaries). In UGX.
manual_discount	INTEGER	NN, DEF 0	Any additional manual discount applied by the Bursar.
net_amount	INTEGER	NN	Computed: standard_amount - bursary_discount - manual_discount.
line_status	VARCHAR(20)	NN, DEF 'pending'	'pending', 'partial', 'paid'.
created_at	TIMESTAMPTZ	NN, DEF now()	

E7. payments

Records every individual payment transaction. One row per payment event. Supports both SchoolPay auto-sync and manual cash/cheque entries.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_bill_id	UUID	FK → pupil_bills.id, NN	The bill this payment is applied to.
amount	INTEGER	NN	Payment amount in UGX.
payment_date	DATE	NN	Date the payment was made.
payment_method	VARCHAR(30)	NN	'schoolpay', 'cash', 'cheque', 'bank_transfer'.
reference_number	VARCHAR(100)	NULL	SchoolPay transaction ref, cheque number, or receipt number.
source	VARCHAR(20)	NN, DEF 'manual'	'manual' (Bursar-entered) or 'schoolpay_sync' (automated).
schoolpay_transaction_id	VARCHAR(100)	UQ, NULL	Unique SchoolPay transaction ID. Used for duplicate detection during sync.

notes	TEXT	NULL	Optional Bursar notes on this payment.
recorded_by	UUID	FK → users.id, NULL	Staff member who recorded this payment. NULL for auto-synced.
created_at	TIMESTAMPTZ	NN, DEF now()	

E8. payment_plans

An instalment plan for a pupil's bill. A plan defines how many instalments the parent has agreed to pay and by when.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_bill_id	UUID	FK → pupil_bills.id, UQ, NN	One plan per bill.
total_instalments	SMALLINT	NN	Number of planned instalments.
notes	TEXT	NULL	Agreed terms or context for this plan.
created_by	UUID	FK → users.id, NULL	
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

E9. payment_plan_instalments

Individual instalment milestones within a payment plan. Each row defines a due date and expected amount.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
payment_plan_id	UUID	FK → payment_plans.id, NN	
instalment_number	SMALLINT	NN	1, 2, 3... in sequence.
due_date	DATE	NN	Expected payment date for this instalment.
expected_amount	INTEGER	NN	Amount expected for this instalment in UGX.
status	VARCHAR(20)	NN, DEF 'upcoming'	'upcoming', 'paid', 'overdue'.

updated_at	TIMESTAMPTZ	NN, DEF now()	Automatically updated when status changes.

E10. fee_statements

Records of generated fee statements. A fee statement is a printable document showing next term's bill and any outstanding balance. Can be generated per pupil or in bulk per class. Attached to report card envelopes.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	
term_id	UUID	FK → terms.id, NN	The term the statement is billing for (next term).
previous_balance	INTEGER	NN, DEF 0	Outstanding balance from the current/previous term.
next_term_total	INTEGER	NN	Total amount billed for the next term.
grand_total	INTEGER	NN	previous_balance + next_term_total.
pdf_path	VARCHAR(500)	NULL	File path of the generated PDF statement.
generated_by	UUID	FK → users.id, NULL	
generated_at	TIMESTAMPTZ	NN, DEF now()	

GROUP F — ACADEMIC PERFORMANCE

assessment_periods · pupil_scores · pupil_term_results · report_cards · report_card_settings · term_requirements

F1. assessment_periods

Represents a specific assessment period instance within a term (e.g. EOT of Term 1 2026). Links an assessment type configuration to a specific term.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
term_id	UUID	FK → terms.id, NN	The term this assessment period belongs to.
assessment_type_id	UUID	FK → assessment_type_configs.id, NN	The type of assessment (BOT, MOT, EOT, etc.).
label	VARCHAR(80)	NN	Display label. E.g. 'End of Term 1, 2026'.
assessment_date	DATE	NULL	Optional date of the examination.
score_entry_open	BOOLEAN	NN, DEF false	True = class teachers can enter scores for this period.
score_entry_locked	BOOLEAN	NN, DEF false	True = scores are locked. Only DOS or Admin can edit.
created_at	TIMESTAMPTZ	NN, DEF now()	
UNIQUE		(term_id, assessment_type_id)	One instance of each assessment type per term.

F2. pupil_scores

Stores a single score for one pupil, for one subject, in one assessment period. This is the atomic unit of academic data. NULL score = missed exam. Zero = sat and scored zero.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	
subject_id	UUID	FK → subjects.id, NN	
assessment_period_id	UUID	FK → assessment_periods.id, NN	

score	SMALLINT	NULL	Raw score. NULL = absent/not sat. 0 = sat and scored zero.
grade_label	VARCHAR(10)	NULL	Computed grade label (e.g. 'D1', 'C3', 'F9'). Stored after computation for report performance.
grade_points	SMALLINT	NULL	Computed grade points (e.g. 1 for D1, 9 for F9). Stored for aggregate calculation.
teacher_remarks	TEXT	NULL	Remarks entered by the class teacher for this subject on the report card.
teacher_initials	VARCHAR(10)	NULL	Teacher initials as shown on the report card.
entered_by	UUID	FK → users.id, NULL	User who first entered this score.
last_edited_by	UUID	FK → users.id, NULL	User who last modified this score. Must be DOS or Admin.
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	
UNIQUE		(pupil_id, subject_id, assessment_period_id)	One score per pupil per subject per period.

> **Score Edit Security** The entered_by column is set on first entry (class teacher). The last_edited_by column is set on any subsequent edit. Application-level RBAC ensures that only DOS and System Admin roles can write to last_edited_by. Class teachers can only write to entered_by on first save.

F3. pupil_term_results

Stores the computed academic results for a pupil for a specific term and assessment period. This table is populated by the grading engine after scores are entered and locked. It caches computed aggregates and divisions so report cards do not require live recalculation.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	
term_id	UUID	FK → terms.id, NN	
assessment_period_id	UUID	FK → assessment_periods.id, NN	

aggregate_points	SMALLINT	NULL	Sum of grade points for included subjects. Lower = better. NULL if not yet computed.
raw_division_label	VARCHAR(10)	NULL	Division before F9 penalty rule. E.g. 'I'.
final_division_label	VARCHAR(10)	NULL	Division after F9 penalty applied. E.g. 'II' (demoted). This is what prints on the report card.
f9_penalty_applied	BOOLEAN	NN, DEF false	True = the F9 in English/Math rule was triggered and division was demoted.
f9_penalty_subject	VARCHAR(120)	NULL	Name of the subject that triggered the F9 penalty.
average_score	NUMERIC(5,2)	NULL	Average score used for ranking in Lower Primary. NULL for Upper Primary.
class_rank	SMALLINT	NULL	Pupil's rank within their stream for this period.
total_pupils_in_stream	SMALLINT	NULL	Total number of pupils in the stream at time of ranking.
promotion_note	TEXT	NULL	E.g. 'PROMOTED TO P.7'. Set by DOS/Admin at end of year.
computed_at	TIMESTAMPTZ	NULL	When the grading engine last computed these results.
computed_by	UUID	FK → users.id, NULL	System user or admin who triggered recomputation.
UNIQUE		(pupil_id, term_id, assessment_period_id)	One result record per pupil per period.

F4. report_card_settings

Single-row configuration table controlling exactly how report cards are generated and printed. Fully configurable by the System Administrator.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	Always a single active configuration row.

show_bot_on_report	BOOLEAN	NN, DEF true	Whether BOT scores appear on the report card.
show_mot_on_report	BOOLEAN	NN, DEF true	Whether MOT scores appear on the report card.
show_eot_on_report	BOOLEAN	NN, DEF true	Whether EOT scores appear on the report card.
average_periods	BOOLEAN	NN, DEF false	True = show averaged scores across periods instead of individual.
show_class_rank	BOOLEAN	NN, DEF true	Whether class rank is printed on the report card.
ranking_format	VARCHAR(30)	NN, DEF 'ordinal'	'ordinal' (1st, 2nd) or 'numeric' (1 OUT OF 44 PUPILS).
show_grade_guide	BOOLEAN	NN, DEF true	Whether the grade guide is printed at the bottom of the card.
show_school_requirements	BOOLEAN	NN, DEF true	Whether the School Requirements for Next Term section is printed.
show_next_term_dates	BOOLEAN	NN, DEF true	Whether next term start dates (Day/Boarding) are printed.
who_can_generate	VARCHAR(20)	NN, DEF 'dos_admin'	'dos_admin' = only DOS and Admin. 'all_teachers' = all class teachers too.
updated_by	UUID	FK → users.id, NULL	
updated_at	TIMESTAMPTZ	NN, DEF now()	

F5. report_cards

Records metadata about each generated report card PDF. The actual PDF is stored in the file system / object storage; this table tracks what was generated and where the file lives.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	
term_id	UUID	FK → terms.id, NN	
assessment_period_id	UUID	FK → assessment_periods.id, NN	The period this report card is for.
report_type	VARCHAR(20)	NN	'midterm' or 'endofterm'.

pdf_path	VARCHAR(500)	NULL	File path or object storage key for the generated PDF.
generated_by	UUID	FK → users.id, NN	DOS or Admin who generated this report card.
generated_at	TIMESTAMPTZ	NN, DEF now()	
UNIQUE		(pupil_id, term_id, assessment_period_id, report_type)	One report card record per pupil per period per type.

F6. term_requirements

Admin-configurable list of items pupils should bring next term. Printed in the 'School Requirements for Next Term' section of the report card.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
term_id	UUID	FK → terms.id, NN	The term this requirements list applies to.
class_id	UUID	FK → classes.id, NULL	NULL = applies to all classes. Non-null = class-specific list.
item_number	SMALLINT	NN	Display order number (1, 2, 3...).
item_description	VARCHAR(300)	NN	E.g. '24 Exercise Books', '12 Pencils', 'Holiday Package'.
created_by	UUID	FK → users.id, NULL	
created_at	TIMESTAMPTZ	NN, DEF now()	

GROUP G — COMMUNICATION

communication_logs · message_templates · demand_notes

G1. communication_logs

The complete, chronological record of every parent interaction initiated by staff regarding a pupil's fees. This is the interaction history visible to all Bursar and Admin staff.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	The pupil this interaction is about.
guardian_id	UUID	FK → guardians.id, NULL	The specific guardian contacted. NULL if logged generically.
channel	VARCHAR(20)	NN	'sms', 'whatsapp', 'call', 'letter', 'demand_note', 'in_person'.
direction	VARCHAR(10)	NN, DEF 'outbound'	'outbound' (school to parent) or 'inbound' (parent called school).
phone_number_used	VARCHAR(30)	NULL	The number contacted. Stored for reference even if guardian record changes later.
message_body	TEXT	NULL	Message text sent (for SMS/WhatsApp). NULL for calls.
call_outcome	VARCHAR(80)	NULL	For calls: 'connected_promised', 'connected_no_commitment', 'no_answer', 'wrong_number'.
delivery_status	VARCHAR(20)	NULL, DEF 'sent'	'sent', 'delivered', 'failed'. Updated by AT delivery webhook.
at_message_id	VARCHAR(100)	NULL	Africa's Talking message ID for delivery tracking.
staff_notes	TEXT	NULL	Internal notes from the staff member on the outcome.
next_followup_date	DATE	NULL	Optional scheduled next follow-up date.
initiated_by	UUID	FK → users.id, NULL	Staff member who initiated or logged this interaction.
created_at	TIMESTAMPTZ	NN, DEF now()	

G2. message_templates

Reusable message templates for SMS and WhatsApp communications. Templates support placeholders like {{pupil_name}}, {{outstanding_amount}}, {{school_name}}.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
name	VARCHAR(120)	NN	Template name. E.g. 'Payment Reminder', 'Balance Overdue'.
channel	VARCHAR(20)	NN	'sms', 'whatsapp', or 'both'.
body	TEXT	NN	Template text with placeholders. E.g. 'Dear {{guardian_name}}, {{outstanding_amount}} is due...'
is_active	BOOLEAN	NN, DEF true	
created_by	UUID	FK → users.id, NULL	
created_at	TIMESTAMPTZ	NN, DEF now()	
updated_at	TIMESTAMPTZ	NN, DEF now()	

G3. demand_notes

Records of generated demand note PDF documents. These are formal printed letters requesting payment of outstanding fees.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
pupil_id	UUID	FK → pupils.id, NN	
pupil_bill_id	UUID	FK → pupil_bills.id, NN	The bill this demand note relates to.
outstanding_amount	INTEGER	NN	Outstanding amount at the time of generation, in UGX.
pdf_path	VARCHAR(500)	NULL	File path of the generated demand note PDF.
generated_by	UUID	FK → users.id, NN	
generated_at	TIMESTAMPTZ	NN, DEF now()	
communication_log_id	UUID	FK → communication_logs.id, NULL	Link to the communication log entry created when this note was generated.

GROUP H — SYSTEM

system_settings · scheduled_job_logs

H1. system_settings

Flexible key-value store for miscellaneous system-wide settings that do not warrant their own table. Used for feature flags, integration configuration references, and UI preferences.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
key	VARCHAR(120)	UQ, NN	Setting key. E.g. 'schoolpay_sync_interval_minutes', 'sms_sender_id', 'pupil_id_prefix'.
value	TEXT	NN	Setting value as string. Application layer casts to correct type.
description	TEXT	NULL	Human-readable explanation of what this setting controls.
updated_by	UUID	FK → users.id, NULL	
updated_at	TIMESTAMPTZ	NN, DEF now()	

H2. scheduled_job_logs

Records execution history of all automated background jobs (SchoolPay sync, overdue instalment flagging, etc.) for monitoring and debugging.

Column	Data Type	Constraints	Description
id	UUID	PK, NN	
job_name	VARCHAR(100)	NN	E.g. 'schoolpay_sync', 'overdue_flag_check'.
started_at	TIMESTAMPTZ	NN	
completed_at	TIMESTAMPTZ	NULL	NULL = still running or failed before completion.
status	VARCHAR(20)	NN	'running', 'success', 'failed'.
records_processed	INTEGER	NULL	Number of records processed in this run.
error_message	TEXT	NULL	Error details if status = 'failed'.
created_at	TIMESTAMPTZ	NN, DEF now()	

5. Key Table Relationships

The following table summarises all primary foreign key relationships, their cardinality, and the referential integrity rule applied at the database level.

From Table	To Table	Cardinality	ON DELETE Rule
users	roles	Many → One	RESTRICT (cannot delete a role with users)
streams	classes	Many → One	RESTRICT
streams	users (teacher)	Many → One	SET NULL (stream remains if teacher removed)
pupils	streams	Many → One	SET NULL (pupil record kept if stream deleted)
pupil_guardians	pupils	Many → One	CASCADE (delete links if pupil deleted)
pupil_guardians	guardians	Many → One	CASCADE
pupil_photos	pupils	One → One	CASCADE (delete photo if pupil soft-deleted)
class_subject_assignments	classes	Many → One	RESTRICT
class_subject_assignments	subjects	Many → One	RESTRICT
class_subject_assignments	terms	Many → One	RESTRICT
subject_section_rules	school_sections	Many → One	RESTRICT
subject_section_rules	subjects	Many → One	RESTRICT
grading_scale_entries	grading_scales	Many → One	CASCADE (delete entries with scale)
division_boundaries	grading_scales	Many → One	CASCADE
fee_structures	fee_categories	Many → One	RESTRICT
fee_structures	classes	Many → One	RESTRICT
fee_structures	terms	Many → One	RESTRICT
pupil_bursaries	pupils	Many → One	RESTRICT
pupil_bursaries	bursary_schemes	Many → One	RESTRICT
pupil_bills	pupils	Many → One	RESTRICT
pupil_bills	terms	Many → One	RESTRICT

bill_line_items	pupil_bills	Many → One	CASCADE (delete items with bill)
payments	pupil_bills	Many → One	RESTRICT (cannot delete billed with payments)
payment_plans	pupil_bills	One → One	CASCADE
payment_plan_instalments	payment_plans	Many → One	CASCADE
fee_statements	pupils	Many → One	RESTRICT
assessment_periods	terms	Many → One	RESTRICT
assessment_periods	assessment_type_configs	Many → One	RESTRICT
pupil_scores	pupils	Many → One	RESTRICT
pupil_scores	subjects	Many → One	RESTRICT
pupil_scores	assessment_periods	Many → One	RESTRICT
pupil_term_results	pupils	Many → One	RESTRICT
pupil_term_results	terms	Many → One	RESTRICT
report_cards	pupils	Many → One	RESTRICT
term_requirements	terms	Many → One	CASCADE
communication_logs	pupils	Many → One	RESTRICT
communication_logs	guardians	Many → One	SET NULL
demand_notes	pupil_bills	Many → One	RESTRICT

6. Indexing Strategy

Indexes are created on all foreign key columns (automatic in Prisma) and on additional columns that are frequently used in WHERE clauses, JOIN conditions, or ORDER BY expressions.

Table & Column(s)	Reason for Index
pupils (lin)	Frequent search by Learner ID Number during registration and record lookup.
pupils (schoolpay_code)	Used on every SchoolPay sync to match incoming payment to a pupil.
pupils (pupil_id_code)	Used in all search bars and API filters.
pupils (stream_id)	Bulk operations per class (billing, score entry, report cards).
pupils (is_active, deleted_at)	All active-pupil queries filter on both these columns.
pupil_bills (pupil_id, term_id)	Unique constraint already creates this. Used in all fees lookups.
pupil_bills (billing_status)	Defaulters report filters by 'partial' and 'pending' status.
payments (pupil_bill_id)	All payment history queries join on this.
payments (schoolpay_transaction_id)	Duplicate detection during SchoolPay sync.
pupil_scores (pupil_id, assessment_period_id)	Score retrieval for report card generation.
pupil_scores (assessment_period_id, subject_id)	Class-wide score entry grid loads by period and subject.
pupil_term_results (pupil_id, term_id)	Report card data retrieval.
communication_logs (pupil_id, created_at DESC)	Interaction history ordered by most recent first.
audit_logs (entity_type, entity_id)	Audit history lookup by record type and ID.
audit_logs (created_at DESC)	System-wide audit log sorted by time.
grading_scales (is_active)	Grading engine always queries the single active scale.
sessions (refresh_token_hash)	Token lookup on every authenticated API request.
sessions (user_id, revoked_at)	Active session check on login and auth middleware.

7. Grading Engine — Business Logic Reference

This section documents the exact sequence of operations performed by the grading engine service when computing a pupil's results for a given assessment period. The engine reads all grading rules from the database and applies them in order.

Step	Logic
1. Load active grading scale	Query grading_scales WHERE is_active = true. Load all entries from grading_scale_entries ordered by display_order.
2. Load division boundaries	Query division_boundaries for the same active grading scale, ordered by display_order.
3. Load subject rules for section	Query subject_section_rules for the pupil's class school_section_id. Identify: (a) subjects included in aggregate, (b) subjects that are F9 penalty triggers.
4. Load pupil scores	Query pupil_scores for this pupil and assessment_period_id. Join with class_subject_assignments to confirm which subjects are assigned to this class this term.
5. Compute grade per subject	For each score: if score IS NULL → grade_label = NULL, grade_points = NULL (missed exam — handled separately). If score IS NOT NULL → match score range against grading_scale_entries to find grade_label and grade_points. Store both in pupil_scores.
6. Compute aggregate	Sum grade_points for all subjects WHERE include_in_aggregate = true in subject_section_rules. This gives the raw aggregate. For Lower Primary: compute average_score = AVG(score) for included subjects instead.
7. Determine raw division	For Upper Primary: match aggregate against division_boundaries to find raw_division_label. For Lower Primary: use same division boundaries applied to average_score.
8. Apply F9 penalty rule	Check: does any included subject with is_penalty_trigger = true have grade_label = 'F9'? If YES: find the next division (one step lower/worse) from division_boundaries. Set final_division_label to the demoted division. Set f9_penalty_applied = true, record the subject name in f9_penalty_subject. If the raw division is already the worst (Ungraded), no further demotion.
9. Compute class rank	For Upper Primary: rank all pupils in the same stream for this period by aggregate_points ASC (lower = better rank). For Lower Primary: rank by average_score DESC (higher = better rank). Set class_rank and total_pupils_in_stream on each pupil_term_results row.
10. Persist results	Write all computed values to pupil_term_results. Update grade_label and grade_points on each pupil_scores row. Set computed_at timestamp.

> **Score Edit After Computation** If a score is edited after the grading engine has run, the engine must be re-triggered for that pupil's full period to recompute grades, aggregate, division, F9 penalty, and rankings. Only DOS and System Admin can trigger score edits after locking.

8. Document Approval

This Database Design Document shall be reviewed and approved before database implementation begins. Approval confirms that the schema accurately reflects all functional requirements and business rules agreed upon during the design phase.

Name	Role	Signature	Date
	School Representative (Client)		
	Lead Developer		
	Director of Studies (DOS)		
	System Administrator		

— End of Document —